

Full Stack Development(3813ICT)

Fabulari Phase 2 Submission

Name: Tariro Kandeya

Student number: s5373405

Workshop: GC Thursday @1100

Repository: <https://github.com/TarieBeverly/fabulari>

Submission branch: main. Phase 2 development branch: codex/phase2.

Phase 1 checkpoint: 5b7c152

MongoDB persistence and Socket.IO community chat

The Phase 1 checkpoint is preserved at commit 5b7c152. This report describes the Phase 2 implementation on main, including MongoDB persistence, real-time community chat and verified local HTTP and HTTPS operation.

Requirements and scope

The elicited requirements and teaching-team clarifications remain the basis of administration permissions. Phase 2 replaces JSON persistence with MongoDB and the local chat preview with authenticated Socket.IO conversations. Super Admins administer the system and do not enter chat rooms. Group creation/deletion and account deletion retain their required request/approval paths.

Members can send plain text and PNG/JPEG/GIF images up to 2 MB, see active room participants and arrivals/departures, and delete their own messages for everyone currently in the room. Only five recent messages per room persist. Active clients keep their session's conversation until leaving; older own messages may be deleted during that session. One account can occupy only one room across tabs. Joining elsewhere closes the previous room. Bans, membership removal, account deletion, room deletion and changed age limits revalidate live access. URLs remain plain text. Video/voice, friend lists, typing indicators and message search are outside the brief.

Architecture and MongoDB

Angular 20 uses AuthService and ApiService for HTTP and ChatService for socket communication. Express handles authentication and approved administration. The existing salted bcrypt password hashes and UUID references are retained during migration. MongoDB collections are users, groups, channels, requests, logs, roomHistory and media. Administrative collections have unique id indexes. roomHistory uses the channel UUID as its MongoDB _id and stores at most five messages. Profile and chat images are stored in server/uploads; the media collection stores metadata. Messages reference authenticated /api/media/:mediaId URLs and record the author's profile-picture URL. Earlier profile pictures remain available to messages until the account is deleted.

Administrative writes run in a MongoDB transaction. A shared queue serializes HTTP and socket mutations in this single server process to prevent overlapping snapshot updates. This design is intended for the assignment's single server; multiple workers would need distributed concurrency control. Sessions remain in express-session's memory store and expire after one hour or a restart. Production deployment would need a persistent session store.

The local development runner uses a real, disk-backed MongoDB replica set with wiredTiger; the database is stored under `server/.mongo/data` and is preserved on shutdown. It downloads the official MongoDB server binary on first use. MongoDB Atlas can instead be supplied through `MONGODB_URI`. Import refuses to overwrite a nonempty database.

HTTP and socket interfaces

Existing administrative HTTP endpoints keep their validation, roles and approval requirements. Session cookies authenticate both HTTP and Socket.IO. Every socket action reloads the session and checks current membership, age and role. Logout disconnects sockets belonging to the ended session. Untrusted browser origins are refused.

Event	Direction	Behaviour
room:join	Client to server	Validate access; replace previous active tab/room; acknowledge recent five messages and participants
room:leave	Client to server	Remove presence and clear session message ownership
message:send	Client to server	Validate text/image; persist latest five; broadcast message:new
message:delete	Client to server	Check own message ID in current room session; update retained history; broadcast message:deleted
room:participants	Server to client	Current participants only
room:notice	Server to client	Arrival/departure notification
room:closed	Server to client	Access removed, replacement tab, or session end
messages:authors	Server to client	Remove deleted users' messages from active clients
workspace:updated	Server to client	Refresh administrative data after saved changes

Acknowledgements use `{ok:true,...}` or `{ok:false,message}`. The client handles timeouts and displays errors. Successful sends broadcast to all active room clients, including the sender. Stored history is available only after authorised room entry.

Design and accessibility

The original wireframes and storyboard were recorded before application coding. Phase 2 keeps the same dashboard layout and replaces preview labels with live rooms, presence and shared-message actions. Forms retain labels, visible focus styles, accessible error/status messages, keyboard buttons and responsive layouts. Chat uses plain text interpolation to avoid executing user HTML. Theme colours come from groups with optional profile overrides.

Automated verification

Verification completed on 8 October 2026: 8 Phase 1 regression tests, 12 HTTP MongoDB/Socket.IO integration tests and 7 Angular client tests passed. The production build passed with a dashboard CSS size warning. The

student confirmed browser checks for shared text, live delivery in both directions, message deletion, image sharing, participant departure updates and logout protection over local HTTP.

Check	Evidence/result
Bootstrap, profiles, registration and role assignment	Existing workflow tests pass
Authentication, wrong passwords, origins and logout	Existing workflow tests pass
Requests, age limits, group/room administration and no-orphan safeguards	Existing workflow tests pass
Real MongoDB administrative persistence	Integration test passes
Two clients, room presence and shared text	Integration test passes
Own deletion across clients, including older active-session messages	Integration test passes
Image signature validation and valid image delivery	Integration test passes
Five retained messages and second-tab replacement	Integration test passes
Approved ban immediately removes live access	Integration test passes
Logout disconnects the existing socket	Integration test passes
Changed age limits, account deletion message cleanup, room-history deletion	Integration test passes
Profile picture replacement	Earlier message pictures remain accessible; regression test passes
HTTPS transport and secure chat	14 checks passed with certificate verification enabled
Angular session and navigation	7 client tests passed
Production Angular build	Passed; dashboard CSS size warning
Browser HTTP and HTTPS evidence	HTTP chat walkthrough passed; HTTPS connection and sign-in passed

HTTPS verification on 8 October 2026: all 14 secure deployment checks passed. Certificate verification remained enabled. Checks covered rejection of an untrusted certificate, TLS 1.2 or later, production Angular page/assets, Secure session cookies, authenticated encrypted WebSocket text and images, permissions, presence, retained history and logout. The dedicated localhost certificate was trusted with student approval. Edge reported "Connection is secure" and demo sign-in opened the HTTPS dashboard. Secure room workflows passed automated tests; the HTTPS browser room walkthrough was not completed because the browser automation tool encountered a window-coordinate error.

Run `npm test` in server for the isolated JSON regression suite. With MongoDB running, run `npm run test:mongo` for the HTTP MongoDB/socket suite. Build the Angular client, then run `npm run test:https` for the secure deployment suite. Integration tests use a randomly named test database and temporary uploads, and remove only their own test data. Run `npm test -- --watch=false --browsers=ChromeHeadless` in client for the Angular tests.

Running and remaining checks

See docs/phase2-startup.md for reproducible setup. HTTP development uses the Angular server on port 4200 and the backend on port 3000. The production Angular build, authenticated API, images and Socket.IO can instead share <https://localhost:3443>. HTTPS startup uses `npm run start:https` in server with `server/.env.https` configured. Local certificates expire after 30 days and are excluded from Git. The teacher/collaborator invitation and course submission remain student actions.

Dependency and deployment notes

The server and production client dependency audits reported zero vulnerabilities. Compatible development-tool fixes were applied. Remaining audit findings concern the Karma development test-tool chain; a forced downgrade was not applied. Local HTTP and HTTPS verification passed. Public hosting has not been tested. Sessions use an in-memory store and administrative writes are serialized within one server process; a scaled public deployment would need a persistent session store and distributed concurrency control.

Raw Phase 2 Markdown

The following is a copy of Phase2.md from the assignment repository.

Fabulari 📄 Phase 2

- **Name:** Tariro Kandeya
- **Student number:** s5373405
- **Workshop:** GC Thursday @1100
- **Repository:** <https://github.com/TarieBeverly/fabulari>
- **Submission branch:** main
- **Phase 2 development branch:** codex/phase2
- **Phase 1 checkpoint:** 5b7c152

Requirements and scope

The [elicited requirements](docs/requirements.md) and teaching-team clarifications remain the basis of administration permissions. Phase 2 replaces JSON persistence with MongoDB and the local chat preview with authenticated Socket.IO conversations. Super Admins administer the system and do not enter chat rooms. Group creation/deletion and account deletion retain their required request/approval paths.

Members can send plain text and PNG/JPEG/GIF images up to 2 MB, see active room participants and arrivals/departures, and delete their own messages for everyone currently in the room. Only five recent messages per room persist. Active clients keep their session's conversation until leaving; older own messages may be deleted during that session. One account can occupy only one room across tabs. Joining elsewhere closes the previous room. Bans, membership removal, account deletion, room deletion and changed age limits revalidate live access. URLs remain plain text. Video/voice, friend lists, typing indicators and message search are outside the brief.

Architecture and MongoDB

Angular 20 uses AuthService/ApiService for HTTP and ChatService for socket communication. Express handles authentication and approved administration. The existing salted bcrypt password hashes and UUID references are retained during migration. MongoDB collections are users, groups, channels, requests, logs and roomHistory. Administrative collections have unique id indexes; roomHistory uses the channel UUID as its MongoDB _id and stores at most five message objects. Profile and chat images are stored in server/uploads. MongoDB's media collection stores their metadata, and messages reference authenticated /api/media/:mediaId URLs. Messages also record the author's profile-picture URL. Only the latest five messages per room are retained in MongoDB.

Administrative writes run in a MongoDB transaction. A shared queue serializes HTTP and socket mutations in this single server process to prevent overlapping snapshot updates. This design is intended for the assignment's single server; multiple workers would need distributed concurrency control. Sessions remain in express-session's memory store and expire after one hour or a restart. Production deployment would need a persistent session store.

The local development runner uses a real, disk-backed MongoDB replica set with wiredTiger; the database is stored under server/.mongo/data and is preserved on shutdown. It downloads the official MongoDB server binary on first use. MongoDB Atlas can instead be supplied through MONGODB_URI. Import refuses to overwrite a nonempty database.

HTTP and socket interfaces

Existing administrative [HTTP endpoints](docs/api-endpoints.md) keep their validation, roles and approval requirements. Session cookies authenticate both HTTP and Socket.IO. Every socket action reloads the session and checks current membership, age and role. Logout disconnects sockets belonging to the ended session. Untrusted browser origins are refused.

Event	Direction	Behaviour
room:join	Client to server	Validate access; replace previous active tab/room; acknowledge recent five messages and participants
room:leave	Client to server	Remove presence and clear session message ownership
message:send	Client to server	Validate text/image; persist latest five; broadcast message:new
message:delete	Client to server	Check own message ID in current room session; update retained history; broadcast message:deleted
room:participants	Server to client	Current participants only
room:notice	Server to client	Arrival/departure notification
room:closed	Server to client	Access removed, replacement tab, or session end
messages:authors	Server to client	Remove deleted users' messages from active clients
workspace:updated	Server to client	Refresh administrative data after saved changes

Acknowledgements use `{ok:true,...}` or `{ok:false,message}`. The client handles timeouts and displays errors. Successful sends broadcast to all active room clients, including the sender. Stored history is available only after authorised room entry.

Design and accessibility

The original [wireframes](docs/design/wireframes.md) and [storyboard](docs/design/storyboard.md) were recorded before application coding. Phase 2 keeps the same dashboard layout and replaces preview labels with live rooms, presence and shared-message actions. Forms retain labels, visible focus styles, accessible error/status messages, keyboard buttons and responsive layouts. Chat uses plain text interpolation to avoid executing user HTML. Theme colours come from groups with optional profile overrides.

Automated verification

Check	Evidence/result
Bootstrap, profiles, registration and role assignment	Existing workflow tests pass
Authentication, wrong passwords, origins and logout	Existing workflow tests pass
Requests, age limits, group/room administration and no-orphan safeguards	Existing workflow tests pass
Real MongoDB administrative persistence	Integration test passes
Two clients, room presence and shared text	Integration test passes
Own deletion across clients, including older active-session messages	Integration test passes
Image signature validation and valid image delivery	Integration test passes
Five retained messages and second-tab replacement	Integration test passes
Approved ban immediately removes live access	Integration test passes
Logout disconnects the existing socket	Integration test passes
Changed age limits, account deletion message cleanup, room-history deletion	Integration test passes

Verification completed on 8 October 2026:

- Phase 1 regression tests: 8 passed.
- MongoDB and Socket.IO integration tests: 12 passed.
- Angular client tests: 7 passed.
- Production build: passed, with a dashboard CSS size warning.
- Browser checks with demo and groupadmin accounts: shared text, live delivery in both directions, live message deletion, image sharing, participant departure updates and logout protection passed.

Local HTTPS application testing completed: all 14 checks passed, including verified TLS certificates, secure session cookies, production Angular assets, encrypted WebSocket text/images, permission changes and logout. Certificate verification remained enabled. The dedicated localhost certificate was trusted with student approval. Edge reported "Connection is secure" and demo sign-in opened the HTTPS dashboard. Secure room workflows passed automated tests; browser room-button automation was limited by the computer tool window-coordinate error. Public hosting has not been tested.

Running and remaining checks

See [Phase 2 startup](docs/phase2-startup.md). HTTP localhost is the development default. The built Angular application can be served with the API and Socket.IO on one HTTPS origin. See the secure startup instructions; `npm run test:https` verifies the full chat workflow over HTTPS against an isolated MongoDB test database. The teacher/collaborator invitation and LMS submission remain student actions.

Dependency and deployment notes

The server and production client dependency audits report zero vulnerabilities. Compatible development-tool fixes were applied. The remaining audit findings concern the Karma development test-tool chain; a forced downgrade was not applied. HTTPS startup is implemented and automated local deployment checks pass. Browser use requires trusting the local certificate or supplying a certificate already trusted by the browser. The production build and browser walkthrough passed on 8 October 2026. Local HTTPS deployment passed automated testing on 8 October 2026; Edge certificate trust and secure sign-in also passed; public hosting has not been tested.